

Splitting Tuples in C# 7 with Examples

In this article, I am going to discuss **Splitting Tuples in C# 7** with Examples. Please read our previous article before proceeding to this article where we discussed [Tuples in C# 7](#) with Examples. The Splitting Tuples in C# is a process of **Splitting a Variable Value into Multiple Parts** and storing each part into a new variable. This is very useful when you are working with Tuples in C# as we know Tuples are going to store multiple values.

Why do we need to Split Tuples in C#?

As we already discussed Tuples provides a lightweight way to retrieve multiple values from a method call. Once you retrieve the tuple, then you need to handle its individual elements. Handling these elements one by one is really a dirty approach. We can overcome this by splitting the Tuples in C#.

Example to Understand Splitting Tuples in C# 7

Let's understand Splitting Tuples in C# with an example. Please have a look at the below example. As you can see in the below code, we are using Tuples to return four values from the **GetEmployeeDetails** method. And further, if you notice within the Main method, we are storing each value of the Tuple in separate variables. The following example code is self-explained, so please go through the comment lines for a better understanding.

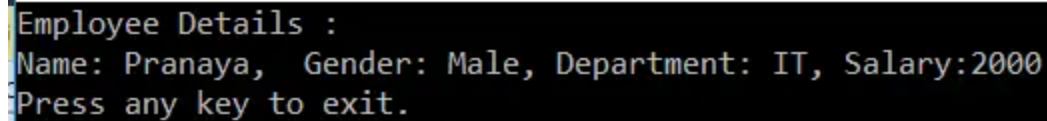
```
using System;
class Program
{
    static void Main()
    {
        //GetEmployeeDetails Method returning a Tuple
        var EmployeeDetails = GetEmployeeDetails(1001);
        //Storing Each Value of Tuple in Separate Variables
        var Name = EmployeeDetails.Item1;
        var Salary = EmployeeDetails.Item2;
        var Gender = EmployeeDetails.Item3;
        var Dept = EmployeeDetails.Item4;
        //Do something with the data.
        //Here we are just printing the data in the console
        Console.WriteLine("Employee Details :");
        Console.WriteLine($"Name: {Name}, Gender: {Gender}, Department: {Dept}
{Salary}");
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    //GetEmployeeDetails() method returns a tuple with 4 values
    private static (string, double, string, string) GetEmployeeDetails(long EmployeeID)
    {
        //Based on the EmployeeID get the data from a database
        //Here we are hardcoded the value
        string EmployeeName = "Pranaya";
        double Salary = 2000;
    }
}
```

```

        string Gender = "Male";
        string Department = "IT";
        return (EmployeeName, Salary, Gender, Department);
    }
}

```

When you run the application, you will get the data as expected as shown below.



```

Employee Details :
Name: Pranaya, Gender: Male, Department: IT, Salary:2000
Press any key to exit.

```

As shown in the above example the **GetEmployeeDetails()** method returns a tuple with 4 values and then we assigned each of its elements to a variable in a separate operation. But from C# 7.0, now we can retrieve multiple elements from a tuple or retrieve multiple fields or property values from a Tuple in a single operation which is called **Splitting Tuples in C#**.

Different Ways to Deconstruct a Tuple or Splitting Tuples in C# 7:

Method 1:

We can explicitly declare the type of each field inside the parentheses. Let's modify the program as shown below to understand this concept. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
class Program
{
    static void Main()
    {
        //De-Constructing Tuples
        (string Name, double Salary, string Gender, string Dept) =
        GetEmployeeDetails(1001);
        //Do something with the data.
        //Here we are just printing the data in the console
        Console.WriteLine("Employee Details :");
        Console.WriteLine($"Name: {Name}, Gender: {Gender}, Department: {Dept}
{Salary}");
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    //GetEmployeeDetails Method returns a tuple with 4 values
    private static (string, double, string, string) GetEmployeeDetails(long EmpID)
    {
        //Based on the EmployeeID get the data from a database
        //Here we are hardcoded the value
        string EmployeeName = "Pranaya";
        double Salary = 2000;
    }
}

```

```

        string Gender = "Male";
        string Department = "IT";
        //Returning 4 Values through a Tuple
        return (EmployeeName, Salary, Gender, Department);
    }
}

```

The above example deconstructs the 4-tuple returned by the GetEmployeeDetails() method explicitly by declaring the types of each field within the parenthesis.

Method 2:

You can also use the var keyword so that C# infers the type of each variable. You can place the var keyword outside of the parentheses. Let us understand this by modifying the code as shown below. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
class Program
{
    static void Main()
    {
        //De-Constructing Tuples
        var (Name, Salary, Gender, Dept) = GetEmployeeDetails(1001);
        //Do something with the data.
        //Here we are just printing the data in the console
        Console.WriteLine("Employee Details :");
        Console.WriteLine($"Name: {Name}, Gender: {Gender}, Department: {Dept}
{Salary}");
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    //GetEmployeeDetails Method returns a tuple with 4 values
    private static (string, double, string, string) GetEmployeeDetails(long EmployeeID)
    {
        //Based on the EmployeeID get the data from a database
        //Here we are hardcoded the value
        string EmployeeName = "Pranaya";
        double Salary = 2000;
        string Gender = "Male";
        string Department = "IT";
        //Returning 4 Values through a Tuple
        return (EmployeeName, Salary, Gender, Department);
    }
}

```

The above example uses type inference when deconstructing the 4-tuple returned by the GetEmployeeDetails method. You can also use the var keyword individually with any or all of the variable declarations inside the parentheses. For a better understanding, please have a look at the below example.

```

using System;

```

```

class Program
{
    static void Main()
    {
        //De-Constructing Tuples
        (var Name, var Salary, string Gender, var Dept) = GetEmployeeDetails(1001);
        //Do something with the data.
        //Here we are just printing the data in the console
        Console.WriteLine("Employee Details :");
        Console.WriteLine($"Name: {Name}, Gender: {Gender}, Department: {Dept}
{Salary}");
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    //GetEmployeeDetails Method returns a tuple with 4 values
    private static (string, double, string, string) GetEmployeeDetails(long EmployeeID)
    {
        //Based on the EmployeeID get the data from a database
        //Here we are hardcoded the value
        string EmployeeName = "Pranaya";
        double Salary = 2000;
        string Gender = "Male";
        string Department = "IT";
        //Returning 4 Values through a Tuple
        return (EmployeeName, Salary, Gender, Department);
    }
}

```

Note: This method is cumbersome and is not recommended.

Method 3:

You may deconstruct the tuple into variables that have already been declared. For a better understanding, please have a look at the below example.

```

using System;
class Program
{
    static void Main()
    {
        //First Declare the Variables
        string Name;
        double Salary;
        string Gender = "Female";
        string Dept = "HR";
        //De-Constructing Tuples into Already Declared variables
        (Name, Salary, Gender, Dept) = GetEmployeeDetails(1001);
        //Do something with the data.
        //Here we are just printing the data in the console
        Console.WriteLine("Employee Details :");
    }
}

```

```

        Console.WriteLine($"Name: {Name}, Gender: {Gender}, Department: {Dept
{Salary}");
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    //GetEmployeeDetails Method returns a tuple with 4 values
    private static (string, double, string, string) GetEmployeeDetails(long EmpID)
    {
        //Based on the EmployeeID get the data from a database
        //Here we are hardcoded the value
        string EmployeeName = "Pranaya";
        double Salary = 2000;
        string Gender = "Male";
        string Department = "IT";
        //Returning 4 Values through a Tuple
        return (EmployeeName, Salary, Gender, Department);
    }
}

```

Points to Remember while Splitting Tuples in C# 7:

Note that you cannot specify a specific type outside the parentheses even if every field in the tuple has the same type. This generates compiler error **CS8136, "Deconstruction 'var (...)' form disallows a specific type for 'var'."**

Note that you must assign each element of the tuple to a variable. If you omit any elements, the compiler generates error **CS8132, "Cannot deconstruct a tuple of 'x' elements into 'y' variables."**

You cannot mix declarations and assignments to existing variables on the left-hand side of a deconstruction. The compiler generates error **CS8184, "a deconstruction cannot mix declarations and expressions on the left-hand side."** when the members include newly declared and existing variables.

In the next article, I am going to discuss the **Local Functions in C# 7** with Examples. Here, in this article, I try to explain **Splitting Tuples in C# 7** with Examples. I hope you enjoy this Splitting Tuple in C# 7 with Examples article.